

Unit - III Knowledge and Reasoning

Teaching Hours: 06

Marks: 12



Knowledge-Based agent

- An Agent who have the capability of maintaining an internal state of knowledge, reason over that knowledge, update their knowledge after observations and take actions.
- These agents can represent the world with some formal representation and act intelligently.
- Knowledge-based agents are composed of two main parts:
 - **Knowledge-base and**
 - **Inference system.**

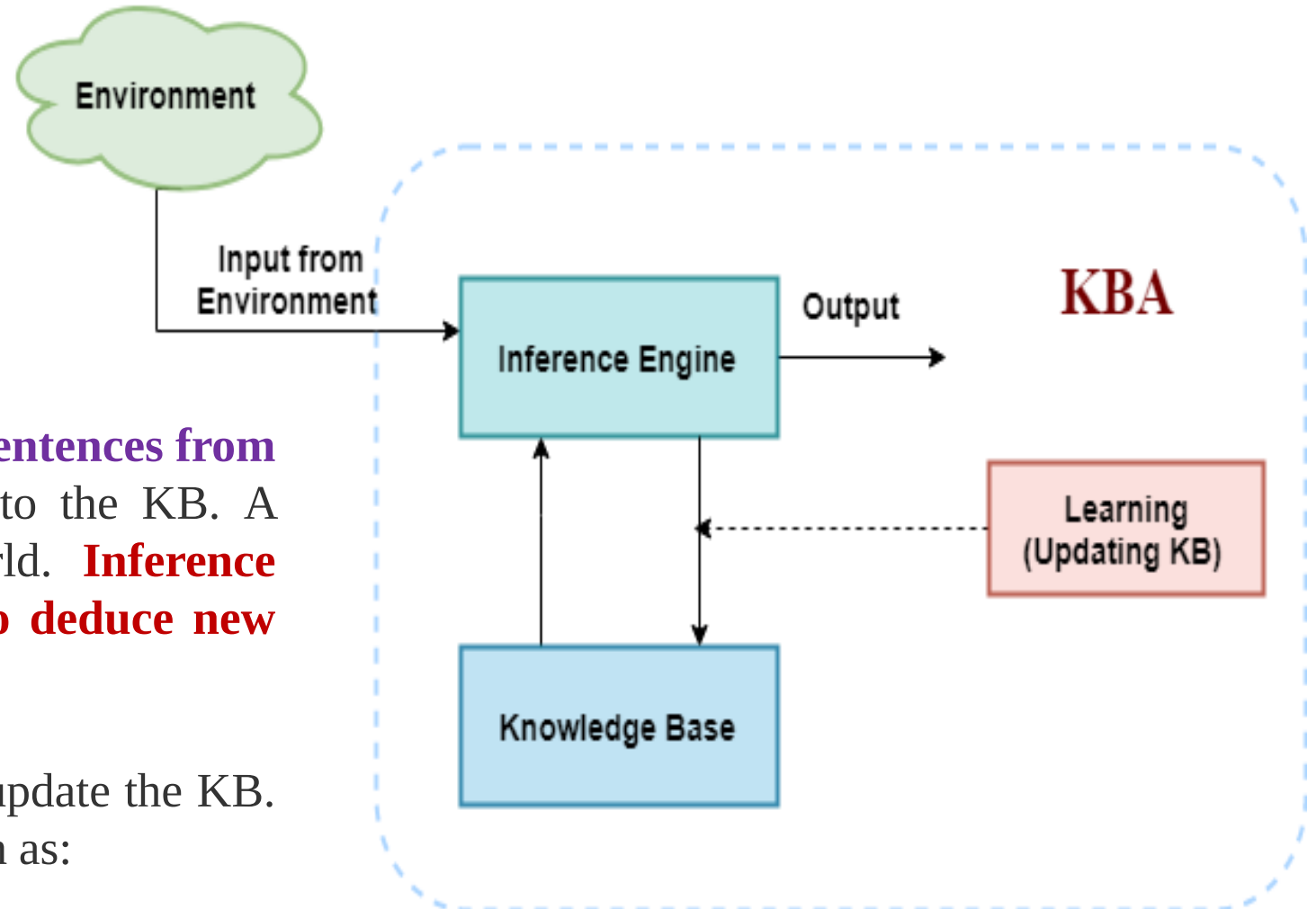
The architecture of knowledge-based agent(KBA):

Knowledge base: It is a central component of a **KBA**, it is also known as KB. It is a **collection of sentences**. These sentences are expressed in a language which is called a knowledge representation language. The Knowledge-base of KBA **stores fact about the world**.

Inference system: It means **deriving new sentences from old**. It allows us to add a new sentence to the KB. A sentence is a proposition about the world. **Inference system applies logical rules to the KB to deduce new information**.

It generates new facts so that an agent can update the KB. It works mainly in two rules which are given as:

- Forward chaining
- Backward chaining



Operations Performed by KBA

- **TELL:** This operation tells the knowledge base what it perceives from the environment.
- **ASK:** This operation asks the knowledge base what action it should perform.
- **Perform:** It performs the selected action.

Various levels of knowledge-based agent

- **1. Knowledge level**

- Knowledge level is the **first level of knowledge-based agent**, in this level, we need to specify what the agent knows, and what the agent goals are.
- Suppose, an automated taxi agent needs to go from a station A to station B, and he knows the way from A to B, so this comes at the knowledge level.

- **2. Logical level:**

- At the logical level, an encoding of knowledge into logical sentences occurs.
- At the logical level we can expect to the automated taxi agent to reach to the destination B.

- **3. Implementation level:**

- At the implementation level agent perform actions as per logical and knowledge level.
- At this level, an automated taxi agent actually implement his knowledge and logic so that he can reach to the destination.

What is knowledge representation?

- Humans are best at understanding, reasoning, and interpreting knowledge.
- Human knows things, which is knowledge and as per their knowledge they perform various actions in the real world.
- **How machines do all these things? It comes under knowledge representation and reasoning.**
- Agent can utilize this knowledge to solve the complex real world problems.
- **Knowledge Representation** is not just storing data into some database, but it also enables an agent to learn from that knowledge.

What to Represent:

- **Object:** All the facts about objects in our world domain. E.g., Guitars contains strings, trumpets are brass instruments.
- **Events:** Events are the actions which occur in our world.
- **Performance:** It describe behavior which involves knowledge about how to do things.
- **Meta-knowledge:** It is knowledge about what we know.
- **Facts:** Facts are the truths about the real world and what we represent.
- **Knowledge-Base:** The central component of the KBA is the knowledge-base. It is represented as KB. The Knowledgebase is a group of the Sentences.

Types of knowledge

- 1. Declarative Knowledge
- 2. Procedural Knowledge
- 3. Meta-knowledge
- 4. Heuristic knowledge
- 5. Structural knowledge

Types of knowledge Conti....

- **1. Declarative Knowledge:**

- Declarative knowledge is to **know about something**.
- It includes **concepts, facts, and objects**.
- It is also called descriptive knowledge and expressed in declarative sentences.
- It is simpler than procedural language.

- **2. Procedural Knowledge**

- It is also known as **imperative knowledge**.
- Procedural knowledge is a type of knowledge which is responsible for knowing **how to do something**.
- It can be directly **applied to any task**.
- It includes **rules, strategies, procedures, agendas**, etc.
- Procedural knowledge depends on the task on which it can be applied.

Types of knowledge Conti....

- **3. Meta-knowledge:**

Knowledge about the other types of knowledge is called Meta-knowledge.

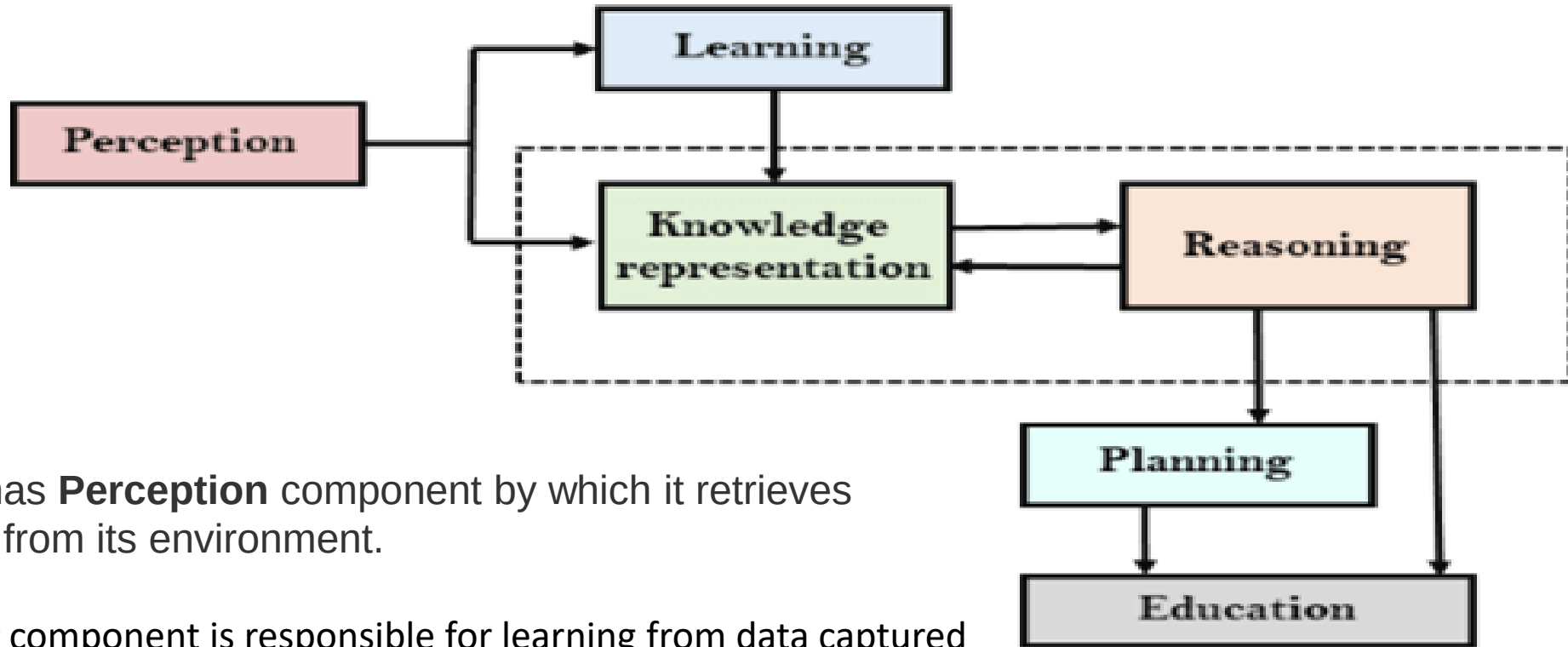
- **4. Heuristic knowledge:**

- Heuristic knowledge is representing **knowledge of some experts in a field or subject**.
- Heuristic knowledge is **rules of thumb based on previous experiences**, awareness of approaches, and which are good to work but not guaranteed.

- **5. Structural knowledge:**

- Structural knowledge is basic knowledge to problem-solving.
- It describes **relationships between various concepts** such as **kind of, part of, and grouping of something**.
- It describes the **relationship** that **exists between concepts or objects**.

AI knowledge cycle:



- AI system has **Perception** component by which it retrieves information from its environment.
- **The learning** component is responsible for learning from data captured by Perception component
- **Knowledge Representation and Reasoning** components are involved in showing the intelligence in machine-like humans.

Approaches to designing a knowledge-based agent

- There are mainly two approaches to build a knowledge

1. Declarative Knowledge:

- We can create a knowledge-based agent by providing facts and rules in the knowledge base instead of programming every action directly.
- designer builds the agent by telling it what it needs to know. The knowledge is stored as facts and rules in the Knowledge Base (KB). The agent then uses reasoning (inference) to decide what action to perform.

2. Procedural Knowledge

- It is also known as **imperative knowledge**.
- Procedural knowledge is a type of knowledge which is responsible for knowing **how to do something**.
- It can be directly **applied to any task**.
- It includes **rules, strategies, procedures, agendas**, etc.
- Procedural knowledge depends on the task on which it can be applied.

NOTE: Today's Modern AI system combine both approaches.

Approaches to knowledge representation:

1. Simple relational knowledge:
2. Inheritable knowledge:
3. Inferential knowledge:
4. Procedural knowledge:

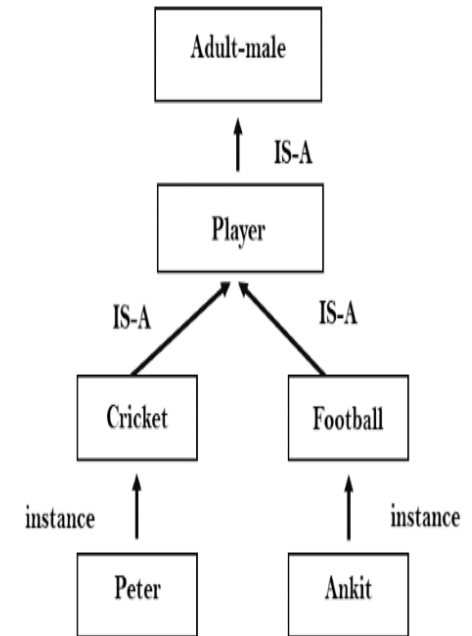
1. Simple relational knowledge:

- It is the simplest way of storing facts which uses the relational method, and each fact about a set of the object is set out systematically in columns.
- This approach is famous in database systems where the relationship between different entities is represented.

Player	Weight	Age
Player1	65	23
Player2	58	18
Player3	75	24

2. Inheritable knowledge:

- In this, all data must be stored into a hierarchy of classes.
- All classes should be arranged in a generalized form or a hierarchal manner.
- This approach contains inheritable knowledge which shows a relation between **instance and class**, and it is called instance relation.
- Every individual frame can represent the collection of attributes and its value.
- **Objects and values** are represented in **Boxed nodes**.
- We use **Arrows which point from objects to their values**.



3. Inferential knowledge:

- Inferential knowledge approach represents knowledge in the form of formal logics.
- This approach can be used to derive more facts.
- It guaranteed correctness.

Example: Let's suppose there are two statements:

- Mohan is a man
- All men are mortal

Then it can represent as;

man(Mohan)

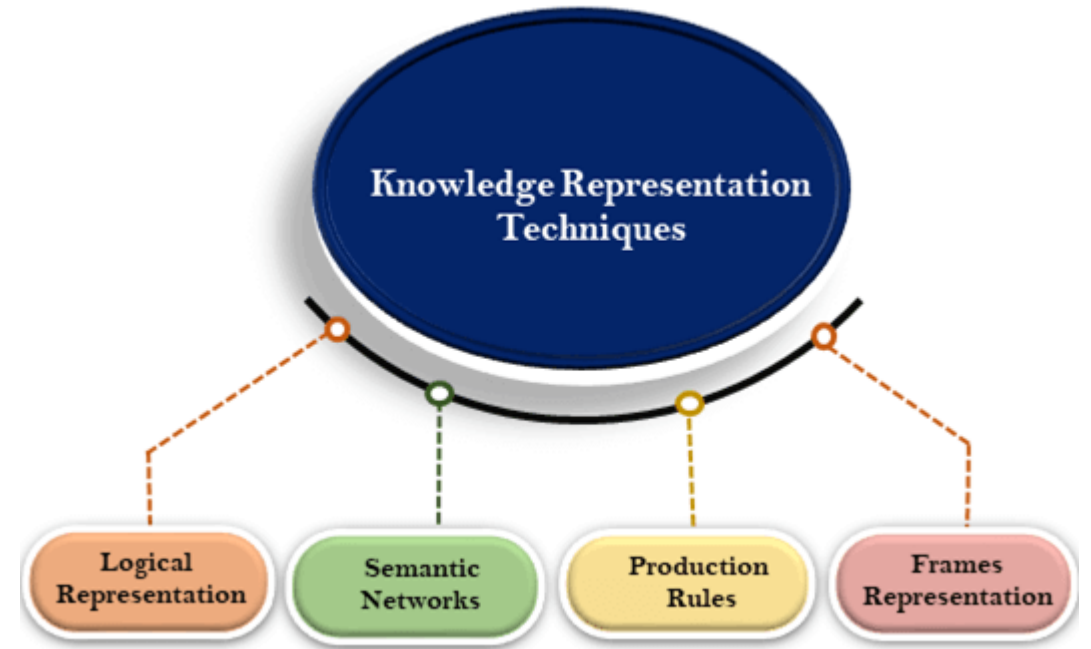
$\forall x = \text{man}(x) \text{ -----} \rightarrow \text{mortal}(x)$

4. Procedural knowledge:

- It **uses small programs and codes** which describes how to do specific things, and how to proceed.
- **If-Then rule** is used here.
- Various coding languages such as **LISP language** and **Prolog language** used.
- Heuristic or domain-specific knowledge easily represented here.
- There is no need to represent all cases.

Techniques of knowledge representation

- Logical Representation
- Semantic Network Representation
- Frame Representation
- Production Rules



Logical Representation

- Logical representation means drawing a conclusion based on various conditions.
- This representation lays down some important communication rules. It consists of precisely defined syntax and semantics.
- Logical representation can be categorized into mainly two logics:
 - Propositional Logics
 - Predicate logics(First Order Logic)

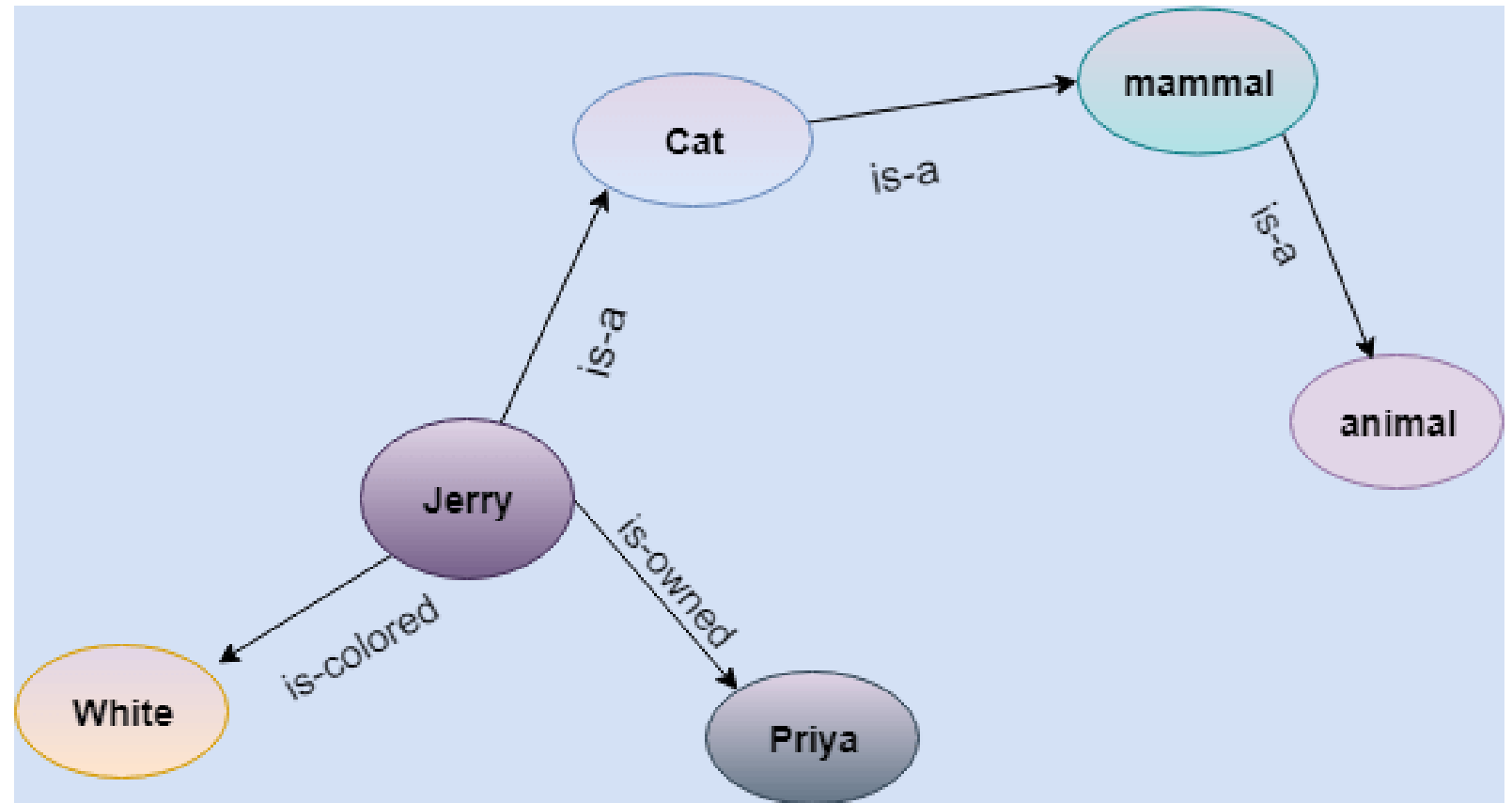
2. Semantic Network Representation

- It is an alternative of predicate logic for knowledge representation.
- Knowledge can represent in the form of graphical networks.
- This network consists of nodes representing objects and arcs which describe the relationship between those objects.
- Semantic networks can categorize the object in different forms and can also link those objects.
- This representation consist of mainly two types of relations:
 - IS-A relation (Inheritance)
 - Kind-of-relation

Example: Following are some statements which we need to represent in the form of nodes and arcs.

Statements:

1. Jerry is a cat.
2. Jerry is a mammal
3. Jerry is owned by Priya.
4. Jerry is White colored.
5. All Mammals are animal.



3. Frame Representation

- A frame is a **record like structure** which consists of a collection of attributes and its values to describe an entity in the world.
- **Frames** are the **AI data structure** which divides knowledge into substructures by representing stereotypes situations.
- It consists of a **collection of slots and slot values**. These slots may be of any type and sizes. Slots have names and values which are called facets.
- **Facets:** The various aspects of a slot is known as **Facets**.
- Facets are features of frames which enable us to put constraints on the frames. Example: IF-NEEDED facts are called when data of any particular slot is needed.
- A frame may consist of any number of slots, and a slot may include any number of facets and facets may have any number of values. A frame is also known as **slot-filter knowledge representation** in artificial intelligence.

Frame for a book

Slots	Filters
Title	Artificial Intelligence
Genre	Computer Science
Author	Peter Norvig
Edition	Third Edition
Year	1996
Page	1152

4. Production Rules

- Production rules system consist of (**condition, action**) pairs which mean, "If condition then action". It has mainly three parts:
 - The set of production rules
 - Working Memory
 - The recognize-act-cycle
- In production rules agent checks for the condition and if the condition exists then production rule fires and corresponding action is carried out.
- The condition part of the rule determines which rule may be applied to a problem.
- And the action part carries out the associated problem-solving steps. This complete process is called a **recognize-act cycle**.
- The working memory contains the description of the current state of problems-solving and rule can write knowledge to the working memory. This knowledge match and may fire other rules.
- If there is a new situation (state) generates, then multiple production rules will be fired together, this is called conflict set. In this situation, the agent needs to select a rule from these sets, and it is called a conflict resolution.

Example:

- IF (at bus stop AND bus arrives) THEN action (get into the bus)
- IF (on the bus AND paid AND empty seat) THEN action (sit down).
- IF (on bus AND unpaid) THEN action (pay charges).
- IF (bus arrives at destination) THEN action (get down from the bus).

Propositional logic

- It is a technique of knowledge representation in logical and mathematical form.
- It is the simplest form of logic where all the statements are made by propositions.
- A proposition is a declarative statement which is either true or false.
- Examples:
 - a) It is Sunday.
 - b) The Sun rises from West (False proposition)
 - c) $3+3=7$ (False proposition)
 - d) 5 is a prime number.

Syntax of propositional logic

The syntax of propositional logic defines the allowable sentences for the knowledge representation. There are two types of Propositions:

- Atomic Propositions
- Compound propositions

•**Atomic Proposition:** Atomic propositions are the simple propositions. It consists of a single proposition symbol. These are the sentences which must be either true or false.

Example:

- a) $2+2$ is 4 , it is an atomic proposition as it is a **true** fact.
- b) "**The Sun is cold**" is also a proposition as it is a **false** fact.

•**Compound proposition:** Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.

Example:

- a) "It is raining today, and street is wet."
- b) "Ankit is a doctor, and his clinic is in Mumbai."

Propositional Logic Connectives

Connective symbols	Word	Technical term	Example
$\wedge$	AND	Conjunction	$A \wedge B$
$\vee$	OR	Disjunction	$A \vee B$
$\rightarrow$	Implies	Implication	$A \rightarrow B$
$\Leftrightarrow$	If and only if	Biconditional	$A \Leftrightarrow B$
$\neg$ or $\sim$	Not	Negation	$\neg A$ or $\neg B$

Propositional Logic Connectives

- **Conjunction:** Rohan is intelligent and hardworking.
P= Rohan is intelligent,
Q= Rohan is hardworking. $\rightarrow P \wedge Q$.
- **Disjunction:** "Ritika is a doctor or Engineer",
Here P= Ritika is Doctor. Q= Ritika is Doctor, so we can write it as $P \vee Q$.
- **Implication:** If it is raining, then the street is wet.
Let P= It is raining, and Q= Street is wet, so it is represented as $P \rightarrow Q$
- **Biconditional:** If I am breathing, then I am alive
P= I am breathing, Q= I am alive, it can be represented as $P \Leftrightarrow Q$.

Tautology:

- A tautology is a compound proposition that is always true, regardless of the truth values of its atomic propositions. In other words, no matter what truth values are assigned to the individual propositions, the overall expression will evaluate to true.

if any compound proposition is always T $\Rightarrow$ Tautology

Tautology:

- Example $P \vee \neg P$
Always True tautology:

P	$\neg P$	$P \vee \neg P$
T	F	T
F	T	T

(always true)

Tautology:

- Example $(P \wedge Q) \rightarrow P$ (Yes) ← Tautology.
when ever it is F then implies is T.

P ✓	Q ✓	$P \wedge Q$	$(P \wedge Q) \rightarrow P$? F
F.	F.	F.	T
F.	T.	F.	T
T.	F.	F.	T
T.	T.	T.	T

Tautology:

- Example $P \rightarrow (Q \rightarrow P) \Rightarrow$ Tautology. ✓

P	Q	$Q \rightarrow P$?	$P \rightarrow (Q \rightarrow P)$ F
T	T.	T	T
T	F	T	T
F	T.	F	T
F	F	T	T

Tautology:

- Example $P \vee (P \wedge Q) \leftrightarrow P$.

P	Q	$(P \wedge Q)$	$P \vee (P \wedge Q)$	P	$P \vee (P \wedge Q) \leftrightarrow P$
F	F	F	F	F	T
F	T	F	F	F	T
T	F	F	T	T	T
T	T	T	T	T	T

Tautology:

- Example $\neg (P \wedge Q) \leftrightarrow (\neg P \vee \neg Q)$

P	Q	$\neg P$	$\neg Q$	$(\neg P \vee \neg Q)$	$\neg (P \wedge Q)$	$\neg (P \wedge Q) \leftrightarrow (\neg P \vee \neg Q)$
F	F	T	T	T	T	T
F	T	T	F	T	T	T
T	F	F	T	T	T	T
T	T	F	F	F	F	T

Contradiction:

- Contradiction in Propositional Logic
- A contradiction is a compound proposition that is always false, regardless of the truth values of its atomic propositions. In other words, no matter what truth values are assigned to the individual propositions, the overall expression will evaluate to false.

→ The proposition which are always False ✓

Contradiction:

- A contradiction is the opposite of a tautology. It reflects a logical impossibility- something that can never be true. Contradictions are useful in logical reasoning to identify when an argument or a set of premises is inherently flawed or inconsistent.

$$(P \wedge (P \rightarrow Q) \wedge \neg Q)$$

$\neg Q$
F
T
F
T

P Q
T T
T F
F F
F T

$P \rightarrow Q$
T
F
T
T

$P \wedge (P \rightarrow Q)$
T
F
F
F

$$(P \wedge (P \rightarrow Q) \wedge \neg Q)$$

F
F
F
F

→ Contradiction

$$(\neg (P \rightarrow Q) \wedge P \wedge Q)$$

P	Q	$(P \rightarrow Q)$	$\neg (P \rightarrow Q) \wedge P \wedge Q$
F	F	T	F
F	T	T	F
T	F	F	F
T	T	T	F

Contingency in Propositional Logic:

- A contingency is a compound proposition that is neither always true (a tautology) nor always false (a contradiction). Its truth value depends on the truth values of its atomic propositions, meaning it can be either true or false based on different assignments of truth values.
- Explanation: Contingent propositions are the most common type of propositions in logic because their truth depends on the specific conditions. Unlike tautologies and contradictions, which have fixed truth values, contingencies vary based on the situation.

Contradiction in Propositional Logic:

- A contradiction is the opposite of a tautology. It reflects a logical impossibility—something that can never be true. Contradictions are useful in logical reasoning to identify when an argument or a set of premises is inherently flawed or inconsistent.

Contingency in Propositional Logic:

- A contradiction is the opposite of a tautology. It reflects a logical impossibility—something that can never be true. Contradictions are useful in logical reasoning to identify when an argument or a set of premises is inherently flawed or inconsistent.
- **Identifying a Contingency Truth Table:** A proposition is a contingency if its truth table contains both true and false outcomes. This means the truth value depends on specific cases. Logical Status: Contingencies are neither universal truths (like tautologies) nor universal falsehoods (like contradictions).

Contingency in Propositional Logic:

- Example 1: $(P \wedge Q)$

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

Examples:

Contradiction

- Contradiction in Propositional Logic
- A contradiction is a compound proposition that is always false, regardless of the truth values of its atomic propositions. In other words, no matter what truth values are assigned to the individual propositions, the overall expression will evaluate to false.

Contradiction

- A contradiction is the opposite of a tautology. It reflects a logical impossibility—something that can never be true. Contradictions are useful in logical reasoning to identify when an argument or a set of premises is inherently flawed or inconsistent.

#Q. $(\neg(P \rightarrow Q) \wedge P \wedge Q)$

Contradiction.

P	Q	$P \rightarrow Q$	$\neg(P \rightarrow Q)$	$P \wedge Q \wedge \neg(P \rightarrow Q)$
F	F	T	F	F
F	T	T	F	F
T	F	F	T	F
T	T	T	F	F

#Q. What is $((P \rightarrow Q) \wedge (Q \rightarrow P))$

$P \leftrightarrow Q$ equivalence Bi conditional

P	Q	$P \rightarrow Q$	$Q \rightarrow P$	$(P \rightarrow Q) \wedge (Q \rightarrow P)$
F	F	T	T	T
F	T	T	F	F
T	F	F	T	F
T	T	T	T	T

First-Order Logic

- In propositional logic, we can only represent the facts, which are either true or false.
- PL is not sufficient to represent the complex sentences or natural language statements.
 - "Some humans are intelligent", or
 - "Sachin likes cricket."
- FOL is sufficiently expressive to represent the natural language statements in a concise way.
- FOL assumes the following things in the world:
 - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits
 - **Relations:** It can be **unary relation such as:** red, round, is adjacent, **or n-any relation such as:** the sister of, brother of, has color, comes between
 - **Function:** Father of, best friend, third inning of, end of,
- FOL has two main parts:
 - **Syntax**
 - **Semantics**

Syntax of First-Order logic

- The syntax of FOL determines which collection of symbols is a logical expression in first-order logic.
- We write statements in short-hand notation in FOL.
- Basic Elements of FOL

Constant	1, 2, A, John, Mumbai, cat,....
Variables	x, y, z, a, b,....
Predicates	Brother, Father, >,....
Function	sqrt, LeftLegOf,
Connectives	$\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$
Equality	$=$
Quantifier	$\forall$, $\exists$

Atomic sentences

- Atomic sentences are the most basic sentences of FOL.
- These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- FOL statements can be divided into two parts:
 - **Subject:** Subject is the main part of the statement.
 - **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.
- We can represent atomic sentences as Predicate (term1, term2, ..., term n).
- **Example:**

Ravi and Ajay are brothers: $\Rightarrow$ Brothers(Ravi, Ajay).



Chinky is a cat: $\Rightarrow$ cat (Chinky).

Universal Quantifier

- Universal quantifier is a **symbol of logical representation**, which specifies that the statement within its range is true for everything or every instance of a particular thing.
- The Universal quantifier is **represented by a symbol $\forall$** , which resembles an inverted A.
- If x is a variable, then $\forall x$ is read as:
 - **For all x**
 - **For each x**
 - **For every x .**
- Example:
 - **All man drink coffee.** $\forall x \text{ man}(x) \rightarrow \text{drink}(x, \text{coffee}).$
It will be read as: **There are all x where x is a man who drink coffee.**

Existential Quantifier

- Existential quantifiers are the type of quantifiers, which express that the **statement within its scope is true for at least one instance of something**.
- It is denoted by the **logical operator $\exists$** , which resembles as inverted E.
- When it is used with a predicate variable then it is called as an existential quantifier.
- In Existential quantifier we always use **AND or Conjunction symbol ($\wedge$)**.
- If x is a variable, then existential quantifier will be $\exists x$ or $\exists(x)$. And it will be read as:
 - There exists a 'x.'
 - For some 'x.'
 - For at least one 'x.'
- Example:

Some boys are intelligent. $\exists x: \text{boys}(x) \wedge \text{intelligent}(x)$

It will be read as: **There are some x where x is a boy who is intelligent.**

Examples:

Eg:

- a) Ravi likes all kind of food.
- b) Apple and chicken are food.
- c) Anything anyone eats and is not killed is food
- d) Ajay eats peanuts and still alive.
- e) Rita eats that Ajay eats.

$$\forall x : \text{food}(x) \longrightarrow \text{likes}(\text{Ravi}, x)$$

$$\text{i) food}(\text{apple}) \quad \text{ii) food}(\text{chicken})$$

$$\forall x \forall y : \text{eats}(x, y) \wedge \neg \text{killed}(x)$$

$$\text{eats}(\text{ajay}, \text{peanuts}) \wedge \text{alive}(\text{ajay})$$

$$\forall c : \text{eats}(\text{Ajay}, c) \rightarrow \text{eats}(\text{Rita}, c)$$

Examples:

1. All birds fly.

$\forall x \text{ bird}(x) \rightarrow \text{fly}(x).$

2. Every man respects his parent.

$\forall x \text{ man}(x) \rightarrow \text{respects}(x, \text{parent}).$

3. Some boys play cricket.

$\exists x \text{ boys}(x) \rightarrow \text{play}(x, \text{cricket}).$

4. Not all students like both Mathematics and Science.

$\neg \forall (x) [\text{student}(x) \rightarrow \text{like}(x, \text{Mathematics}) \wedge \text{like}(x, \text{Science})].$

5. Only one student failed in Mathematics.

$\exists (x) [\text{student}(x) \rightarrow \text{failed}(x, \text{Mathematics}) \wedge \forall (y) [\neg(x=y) \wedge \text{student}(y) \rightarrow \neg \text{failed}(y, \text{Mathematics})].$

Examples:

#Q. Which of the following formulas is a formalization of the sentence: "Every connected and circuit-free graph is a tree"
Where $\text{cir}(x)$ means "x has circuit".

$\neg \text{Cir}(x)$ ✓

- A** $\forall x [(\text{conn}(x) \wedge \neg \text{cir}(x)) \rightarrow \text{tree}(x)]$
- B** $\exists \neg x [\text{conn}(x) \wedge \neg \text{cir}(x)] \wedge \neg \text{tree}(x)$
- C** $\forall x [(\text{conn}(x) \wedge \neg \text{cir}(x)) \wedge \text{tree}(x)]$
- D** $\forall x [(\text{conn}(x) \wedge \text{cir}(x)) \rightarrow \text{tree}(x)]$

Examples:

#Q. We use the dictionary:

M domain of men

s Sheenu

N(x) x is nice

L(x, y) x loves y

Which of the following formulas corresponds to the sentence:

There is a nice man who loves Sheenu.

(A) $\exists x \in M [N(x) \wedge L(x, s)]$

(B) $\exists x \in M [N(x) \rightarrow L(x, s)]$

(C) $\forall x \in M [N(x) \wedge L(x, s)]$

(D) $\forall x \in M [N(x) \rightarrow L(x, s)]$

Examples:

#Q. Assume the domain is $\mathbb{R}$. Which of the following statements are true?

- I. $\forall x \exists y (xy = 1)$
- II. $\exists x \forall y (x^2 \geq y^2)$
- III. $\exists x \exists y (x^2 \geq y^2)$

false

$$\forall x \exists y P(x, y) \Rightarrow \exists y \forall x P(x, y)$$

at least one y for all x

$$\bullet \forall x \exists y (xy = 1)$$

for all x there exist at least one y ($xy = 1$) Yes

A Only (ii).

$$\exists x \forall y (x^2 \geq y)$$

at least one x satisfy condition for all y. False

B Only (iii).

$$\exists x \exists y (x^2 \geq y) \Rightarrow \text{to make it true only one pair of } x, y$$

(2, 1)

C Only (i) and (ii).

D Only (i) and (iii).

Free and Bound Variables

Free Variable:

A variable is said to be a free variable in a formula if it occurs outside the scope of the quantifier.

Example:

$\forall x \exists (y)[P(x, y, z)]$, where z is a free variable.

Bound Variable:

A variable is said to be a bound variable in a formula if it occurs within the scope of the quantifier.

Example:

$\forall x [A(x) B(y)]$, here x and y are the bound variables.